Documentation: picoCTF "Crack the Gate" Solution

Objective: Bypass a secure login portal using source code analysis and manual HTTP header manipulation.

1. Initial Reconnaissance (Finding the Leak)

Security starts with what is visible. Most web challenges contain "developer breadcrumbs."

- **Action:** Open the challenge URL and view the page source (Ctrl + U).
- **Discovery:** Look for HTML comments (text between ``). In this lab, a comment contained an encoded string.

2. Decoding the "Backdoor"

The string found was encoded using **ROT13**, a simple substitution cipher that replaces a letter with the 13th letter after it in the alphabet.ROT13 Cipher Visual Guide

The ROT13 cipher works by substituting a letter with the 13th letter after it in the alphabet. It is its own inverse, meaning applying ROT13 to an encrypted message decrypts it.

A	B	C	D
N	O	P	Q

Example:

- 'A' becomes 'N'
- 'P' becomes 'C'
- 'GRFG' (Encoded) becomes 'TEST' (Decoded)

- **The Cipher:** ABGR: Wnpx - grzcbenel olcnff: hfr urnqre "K-Qri-Npprff: lrf"
- **The Plaintext:** NOTE: Jack - temporary bypass: use header "X-Dev-Access: yes"

- **Lesson:** Developers sometimes leave bypasses for testing but forget to remove the notes or the functionality before the site goes live.

3. Understanding the Attack Vector

A standard login sends an email and a password. However, the "backdoor" we found allows access if a specific **HTTP Header** is present. We cannot add custom headers through a normal browser login form, so we must use a tool like curl or a Python script.

4. Executing the Attack (Terminal Command)

Using the picoCTF terminal, we craft a request that includes both the required login data and the secret bypass header.

Command:

Bash
```
curl -X POST http://amiable-citadel.picoctf.net:53491/login \
    -H "Content-Type: application/json" \
    -H "X-Dev-Access: yes" \
    -d '{"email":"ctf-player@picoctf.org", "password":"password123"}'
```

Component	Value
Method	POST
URL	http://amiable-citadel.picoctf.net:53491/login
Header 1	Content-Type: application/json
Header 2	X-Dev-Access: yes
Data (JSON)	{"email":"ctf-player@picoctf.org", "password":"password123"}
Command	curl -X POST http://amiable-citadel.picoctf.net:53491/login -H "Content-Type: application/json" -H "X-Dev-Access: yes" -d '{"email":"ctf-player@picoctf.org", "password":"password123"}'

Documentation: picoCTF "Crack the Gate" Solution - The Developer's Oversight

Objective: We cracked a secure login page not by guessing the password, but by finding a secret developer shortcut hidden in the code and using it to walk right in.1. The Sneaky Peek (Finding the Hidden Note)

Think of web security documentation as a map, and the first step is always checking the immediate surroundings. Developers often leave little notes—like breadcrumbs—in the code, especially during testing.

- **What i did:** We went to the challenge website and immediately viewed the source code (the raw text behind the page).
- **What iThe Day I Bypassed a "Secure" Login Gate with a Simple HTML Comment**

I always talk about zero-day exploits, but sometimes, the biggest security risk is simple human oversight.

I recently tackled the picoCTF challenge, **"Crack the Gate,"** and the solution wasn't a complex brute-force attack—it was **finding a note a developer forgot to delete.**

The Vulnerability: Developer Breadcrumbs

The first rule of security is reconnaissance. Here's what happened:

- **Action:** I didn't try to log in; I hit Ctrl + U (View Page Source).
- **Discovery:** Tucked away in the HTML was a comment containing a bizarre, jumbled string of text. Developers often leave notes for themselves. This one was encoded.

The Secret Key (A ROT13 Oopsie)

The jumbled text was encoded with **ROT13**, a simple substitution cipher.

Encoded String	ABGR: Wnpx - grzcbenel olcnff: hfr urnqre "K-Qri-Npprff: lrf"
Decoded Secret	**NOTE: Jack - temporary bypass: use header "X-Dev-Access: yes"**

The Lesson: A developer named "Jack" had left a **temporary bypass header** in the live code and forgot to remove the comment *or* the underlying functionality. This was my ticket in.

Executing the Attack: When a Browser Isn't Enough

A normal login form doesn't let you send custom HTTP Headers. This is where tools come in. To trigger the bypass, I had to craft a custom request using curl:

1. **POST Request:** Send data to the login endpoint.
2. **Fake Credentials:** I included placeholder email/password (the site didn't care).
3. **The Magic Header:** I added -H "X-Dev-Access: yes".

The server's logic saw that specific header and said, **"Oh, a developer! Skip the password check and let them right in."** I immediately received the flag, bypassing the entire authentication mechanism.

The Takeaway for Engineers & Security Pros

This isn't just a CTF trick; it's a real-world reminder of critical security hygiene:

1. **Audit the Obvious:** Always check public-facing source code for comments, hidden fields, or debug paths before launch.
2. **Remove or Disable Backdoors:** Any temporary testing functionality must be explicitly removed or disabled in production environments.
3. **Never Assume Security Through Obscurity:** Encoding a note (like using ROT13) is NOT security. If a vulnerability is visible, it's exploitable.

What's the most unexpected oversight you've found that led to a vulnerability? Share your stories below!

- #picoCTF #CyberSecurity #WebSecurity #AppSec #SecurityAwareness #DeveloperMistakes **found:** Tucked away in an HTML comment was a bizarre, jumbled string of letters.

2. Translating the **The Code:** ABGR: Wnpx - grzcbenel olcnff: hfr urnqre \"K-Qri-Npprff: lrf\"

- **The Translation (The Secret):** NOTE: Jack - temporary bypass: use header \"X-Dev-Access: yes\"
- **The Lesson Learned:** Someone named Jack left a backdoor for testing, maybe forgetting to take it out before launching the site. This is our ticket in!

3. Understanding the Ticket (The Attack Vector)

Normally, a website only accepts a username and password. But this secret note tells us the site has a special door that opens if we send a specific **HTTP Header** along with our login attempt.

A normal browser won't let us attach a secret code like \"X-Dev-Access: yes\" to a standard login click. We need a specialized tool—like curl (a command-line tool for sending data to servers).4. Sending the Magic Knock (Executing the Attack)

Using the terminal, we assembled a perfect request: we told the server we were trying to log in (using random credentials), but we secretly added the special developer pass-phrase.

The Command in Plain English:"Hey server, I'm making a **POST** request to your login page. I'm sending you some JSON data (a fake email and password), AND—most importantly—I'm adding the secret handshake header: X-Dev-Access: yes."

Component	What It Means
Method (POST)	We're sending data to the server, not just asking for a page.
URL	The address of the login gate.
Header (Content-Type)	Standard formality: telling the server the data is in JSON format.
Secret Header	X-Dev-Access: yes - The actual key that unlocks the bypass.
Data	The placeholder email and password.

Why it worked: The server's underlying logic saw the special X-Dev-Access: yes header and immediately said, "Oh, a developer! Skip the password check and let them in."5. The Grand Prize

Because we successfully triggered the developer bypass, the server didn't bother validating the credentials. Instead, it instantly responded with the successful login confirmation, which included the picoCTF flag (the ultimate goal).

- **The Result:** A message confirming success and revealing the flag: {\"success\": true,

\"flag\": \"picoCTF{...}\"}

-----Key Takeaways for Future Challenges

Whenever you hit a login screen in a CTF:

1. **Be a Code Detective:** Always check the source code for comments, hidden fields, or stray notes.
2. **Look for Scrambled Letters:** If the text is unreadable but looks structured (like ROT13 or Base64), try decoding it. That's often where the secret is hidden.
3. **Find the Magic Key:** Figure out what *one thing* the application is looking for to bypass security (a special header, a unique cookie, etc.).
4. **Use the Right Tool:** If a regular browser can't send the secret message, use curl or a scripting language like Python to craft the perfect, custom request.

The user wants to "humanize" the existing technical documentation (picoCTF "Crack the Gate" Solution) and reformat it for a LinkedIn post.

The current document is a detailed, slightly redundant technical walkthrough. To make it suitable for LinkedIn, the tone should be more engaging, focus on the "lesson learned" (the developer oversight), and structure it for quick consumption (using a hook, bullet points, and a strong conclusion/call-to-action).

Goal: Transform the technical solution into a digestible, engaging, and professional LinkedIn post.
Tone: Professional, informative, slightly punchy, and focused on security lessons.
Structure: Hook -> Problem -> Solution (The "Aha!" Moment) -> Technical Detail (Minimal) -> Key Takeaway.

The Day I Bypassed a "Secure" Login Gate with a Simple HTML Comment

We all talk about zero-day exploits, but sometimes, the biggest security risk is simple human oversight.

I recently tackled the picoCTF challenge, **"Crack the Gate,"** and the solution wasn't a complex brute-force attack—it was **finding a note a developer forgot to delete.**

The Vulnerability: Developer Breadcrumbs

The first rule of security is reconnaissance. Here's what happened:

- **Action:** I didn't try to log in; I hit Ctrl + U (View Page Source).
- **Discovery:** Tucked away in the HTML was a comment containing a bizarre, jumbled string of text. Developers often leave notes for themselves. This one was encoded.

The Secret Key (A ROT13 Oopsie)

The jumbled text was encoded with **ROT13**, a simple substitution cipher.

Encoded String	ABGR: Wnpx - grzcbenel olcnff: hfr urnqre "K-Qri-Npprff: lrf"
Decoded Secret	**NOTE: Jack - temporary bypass: use header "X-Dev-Access: yes"**

The Lesson: A developer named "Jack" had left a **temporary bypass header** in the live code and forgot to remove the comment *or* the underlying functionality. This was our ticket in.

Executing the Attack: When a Browser Isn't Enough

A normal login form doesn't let you send custom HTTP Headers. This is where tools come in. To trigger the bypass, we had to craft a custom request using curl:

1. **POST Request:** Send data to the login endpoint.
2. **Fake Credentials:** We included placeholder email/password (the site didn't care).
3. **The Magic Header:** We added -H "X-Dev-Access: yes".

The server's logic saw that specific header and said, **"Oh, a developer! Skip the password check and let them right in."** We immediately received the flag, bypassing the entire authentication mechanism.

The Takeaway for Engineers & Security Pros

This isn't just a CTF trick; it's a real-world reminder of critical security hygiene:

1. **Audit the Obvious:** Always check public-facing source code for comments, hidden fields, or debug paths before launch.
2. **Remove or Disable Backdoors:** Any temporary testing functionality must be explicitly removed or disabled in production environments.
3. **Never Assume Security Through Obscurity:** Encoding a note (like using ROT13) is NOT security. If a vulnerability is visible, it's exploitable.

What's the most unexpected oversight you've found that led to a vulnerability? Share your stories below!

#picoCTF #CyberSecurity #WebSecurity #AppSec #SecurityAwareness #DeveloperMistakes

Why this works:

- -X POST: We are submitting data to the server.
- -H "X-Dev-Access: yes": This tells the server "I am a developer," triggering the bypass logic.
- -d: This sends the JSON data payload that the login script expects.

5. Analyzing the Result

Upon successful execution, the server skips the password validation and returns a JSON response.

- **Success Response:** {"success": true, "flag": "picoCTF{...}"}

Summary for Future Labs

When you encounter a "Gate" or Login challenge:

1. **Always check the source code** for comments or hidden scripts.
2. **Look for encoding** (ROT13, Base64). If it looks like gibberish but has structure, try decoding it.
3. **Identify the "Magic Key"**—is it a specific cookie, a hidden field, or a custom header?

Use curl or Python to send a modified request that the standard browser won't let you send.Secret Code